Data Structures and Algorithms 2

Question 2: Minimum Spanning Tree

GitHub repository: https://github.com/janadatatech/dsa2-coursework

Coursework Question 2: Minimum Spanning Tree (25 marks)

Code and test one of the Minimum Spanning Tree (MST) algorithms, either Prim's or Kruskal's, for a connected NetworkX graph with 6 nodes and 11 edges. You must use the following signature:

```
def MyMinimumSpannigTree(Graph) -> Graph
```

This tells you that your code should contain a function called "MyMinimumSpanningTree" which receives 1 argument in the form of a NetworkX graph, and it returns 1 NetworkX graph.

Coding

The steps that you must do are below:

a) Create and depict the original connected graph using matplotlib.

b) Call the MyMinimumSpanningTree function.

- State which MST algorithm you are following.
- Show the creation of the MST step by step using the console.

c) Depict the final MST using matplotlib.

Testing

a) Do the same for 2 new graphs that you create. The graphs must be connected and have at least 5 nodes and 8 edges, and be different to the first one and between themselves.

To summarize: in your report you should have images of 3 different graphs with their correspondent 3 different MSTs.

Record yourselves explaining the MST algorithm that you selected, citing your sources. Then, explain the rationale of your code and include a clip of your code running and building the MST step by step for the first graph.

Q2(a): Create and display the original graph

Plan is to use NetworkX to store the graph and Matplotlib to display it. The graph will have 6 nodes

and 11 weighted edges.

```
import networkx as nx
import matplotlib.pyplot as plt
```

```
graph = nx.Graph()
edges = [
    ("A", "B", 4),
    ("A", "C", 2),
    ("A", "D", 7),
    ("B", "C", 3),
    ("B", "D", 5),
    ("B", "E", 8),
    ("C", "D", 6),
    ("C", "E", 9),
    ("D", "E", 1),
    ("D", "F", 10),
    ("E", "F", 11)
]
for node1, node2, weight in edges:
    graph.add_edge(node1, node2, weight=weight)
print("Number of nodes:", graph.number_of_nodes())
print("Number of edges:", graph.number_of_edges())
print("Graph is connected:", nx.is_connected(graph))
```

```
Number of nodes: 6
Number of edges: 11
Graph is connected: True
```

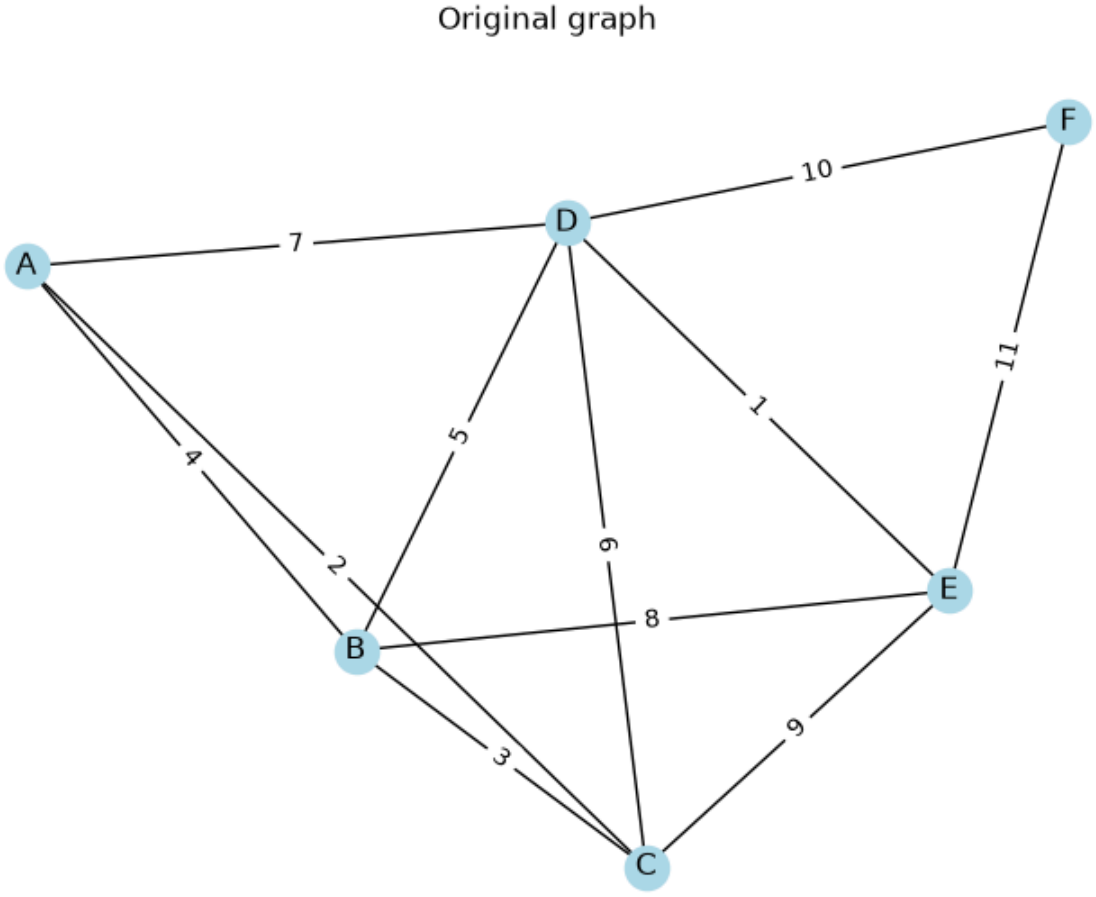
Display the original graph

Each edge is stored as a tuple containing two nodes and a weight and a for loop adds the edges to the graph.

This checks and confirms that the graph is connected and has 6 nodes and 11 edges which I can then display it with its node labels and edge weights.

```
positions = nx.spring_layout(graph, seed=1)
plt.figure(figsize=(7, 5))
nx.draw(graph, positions, with_labels=True, node_color="lightblue")
weights = nx.get_edge_attributes(graph, "weight")
nx.draw_networkx_edge_labels(graph, positions, edge_labels=weights)

plt.title("Original graph")
plt.axis("off")
plt.show()
```


Original graph
A
B
C
D
E
F
7
10
4
2
5
6
1
11
8
3
9

Each labelled circle represents a node where the lines represent edges, and the numbers show their weights.

I save the node positions in "positions" so the minimum spanning tree can use the same layout.

Q2(b): Build the minimum spanning tree

I use Prim's algorithm to build the tree, starting from one node where the list tracks the nodes already included and the loops check the edges connecting these nodes to nodes outside the tree (Moros, no date).

An if statement selects the edge with the smallest weight and the selected edge adds a new node without creating a cycle, repeating until all nodes are included.

For the first graph, the tree will contain 6 nodes and 5 edges

```
def MyMinimumSpanningTree(Graph):
    tree = nx.Graph()
    nodes = list(Graph.nodes())
    start = nodes[0]
    visited = [start] # Track nodes already in the tree.
    tree.add_node(start)

    print("Starting node:", start)

    while len(visited) < len(nodes):     # Keep adding edges until all nodes included.
        smallest_weight = None
        for node in visited:
            for neighbour in Graph.neighbors(node):
                if neighbour not in visited:
                    weight = Graph[node][neighbour]["weight"]
                    if smallest_weight is None or weight < smallest_weight:
                        smallest_weight = weight
                        from_node = node
                        to_node = neighbour

        tree.add_edge(from_node, to_node, weight=smallest_weight) # Add the selected edge and
its new node.
        visited.append(to_node)

        print("Added edge:", from_node, "-", to_node,
              "Weight:", smallest_weight)

    return tree
```

Run Prim's algorithm

I pass the graph into the function and store the returned tree in "mst". The printed output shows the starting node and each selected edge with its weight.

```
mst = MyMinimumSpanningTree(graph)
```

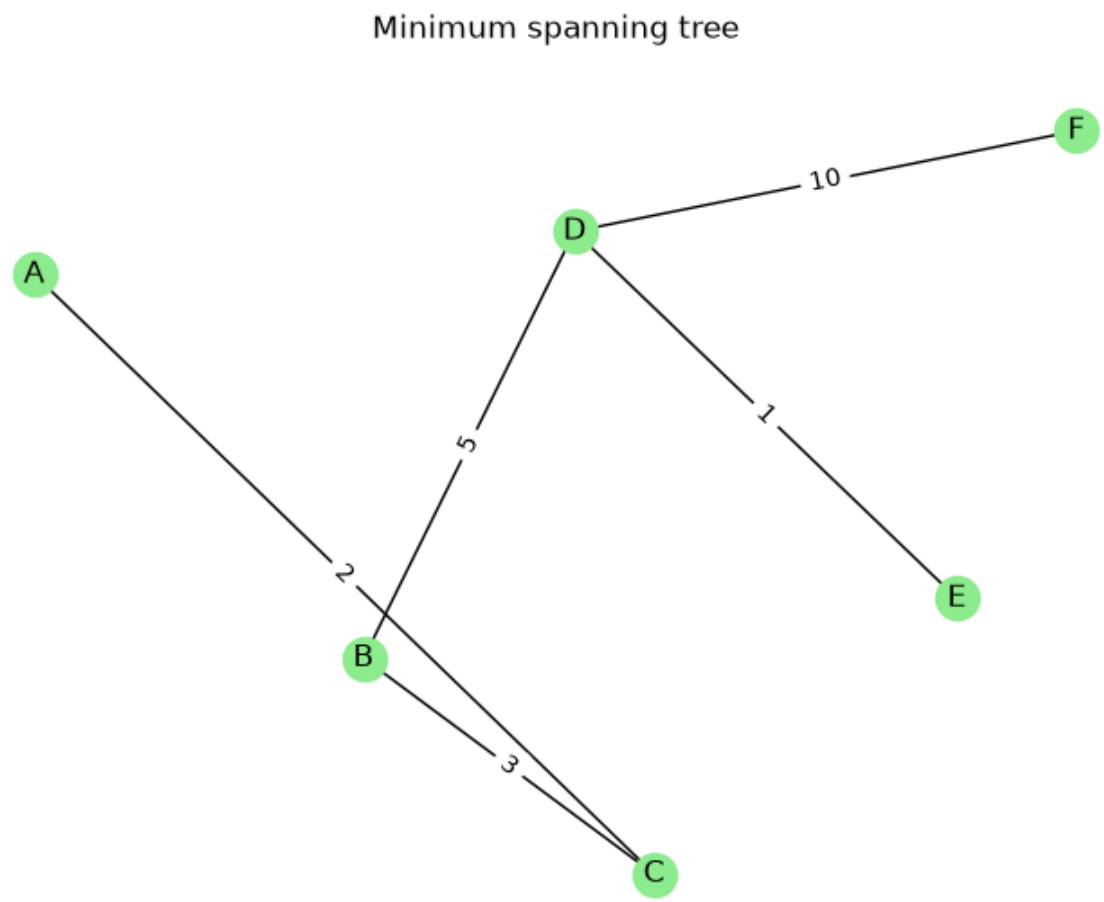
```
Starting node: A
Added edge: A - C Weight: 2
Added edge: C - B Weight: 3
Added edge: B - D Weight: 5
Added edge: D - E Weight: 1
Added edge: D - F Weight: 10
```

Q2(c): Display the minimum spanning tree

The diagram shows the edges selected by Prim's algorithm and their weights.

I keep the nodes in the same positions as the original graph so the two diagrams are easier to compare.

```
plt.figure(figsize=(7, 5))
nx.draw(mst, positions, with_labels=True, node_color="lightgreen")
weights = nx.get_edge_attributes(mst, "weight")
nx.draw_networkx_edge_labels(mst, positions, edge_labels=weights)
plt.title("Minimum spanning tree")
plt.axis("off")
plt.show()
```



I check that the tree contains all 6 nodes and has 5 edges and also check that it is connected and has no cycles.

A for loop adds the edge weights together which gives expected total weight of 21.

```
print("Number of nodes:", mst.number_of_nodes())
print("Number of edges:", mst.number_of_edges())
print("Is a tree:", nx.is_tree(mst))
total_weight = 0

for node1, node2 in mst.edges():
    weight = mst[node1][node2]["weight"]
    total_weight = total_weight + weight
print("Total weight:", total_weight)
```

```
Number of nodes: 6
Number of edges: 5
Is a tree: True
Total weight: 21
```

Testing: Graph 2

I test the same Prim's algorithm function on a different graph with 5 nodes and 8 weighted edges.

A list of tuples stores the edges and a for loop adds each edge and its weight to the graph.

```
graph2 = nx.Graph()
edges2 = [
    ("A", "B", 6),
    ("A", "C", 3),
    ("A", "D", 8),
    ("B", "C", 2),
    ("B", "D", 5),
    ("B", "E", 7),
    ("C", "E", 4),
    ("D", "E", 1)
]
for node1, node2, weight in edges2:
    graph2.add_edge(node1, node2, weight=weight)
print("Number of nodes:", graph2.number_of_nodes())
print("Number of edges:", graph2.number_of_edges())
print("Graph is connected:", nx.is_connected(graph2))
```

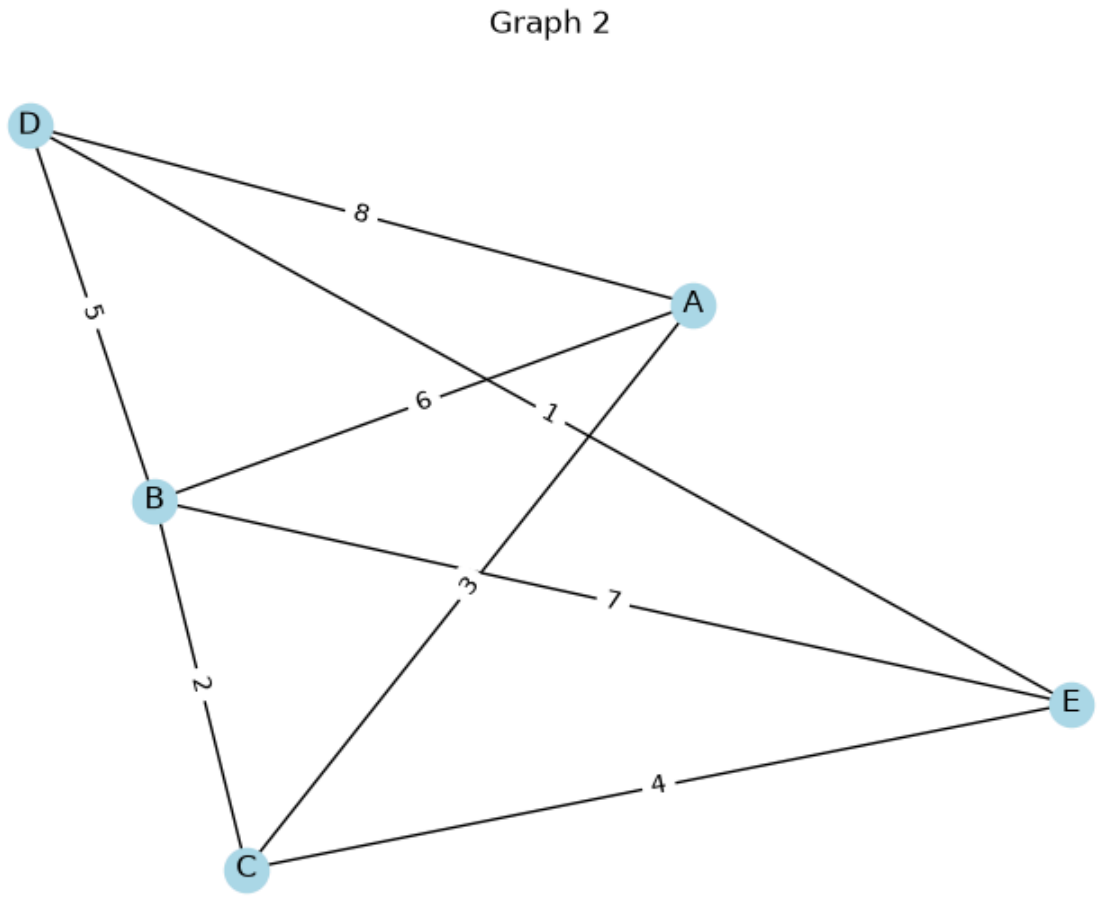
```
Number of nodes: 5
Number of edges: 8
Graph is connected: True
```

Display Graph 2

The diagram shows the second graph and its edge weights.

I save the node positions in `positions2` so its minimum spanning tree can use the same layout.

```
positions2 = nx.spring_layout(graph2, seed=1)
plt.figure(figsize=(7, 5))
nx.draw(graph2, positions2, with_labels=True, node_color="lightblue")
weights2 = nx.get_edge_attributes(graph2, "weight")
nx.draw_networkx_edge_labels(graph2, positions2, edge_labels=weights2)
plt.title("Graph 2")
plt.axis("off")
plt.show()
```



Run Prim's algorithm on Graph 2

I call the same function using "graph2" and the result is stored in "mst2" where the function prints each edge it adds and its weight.

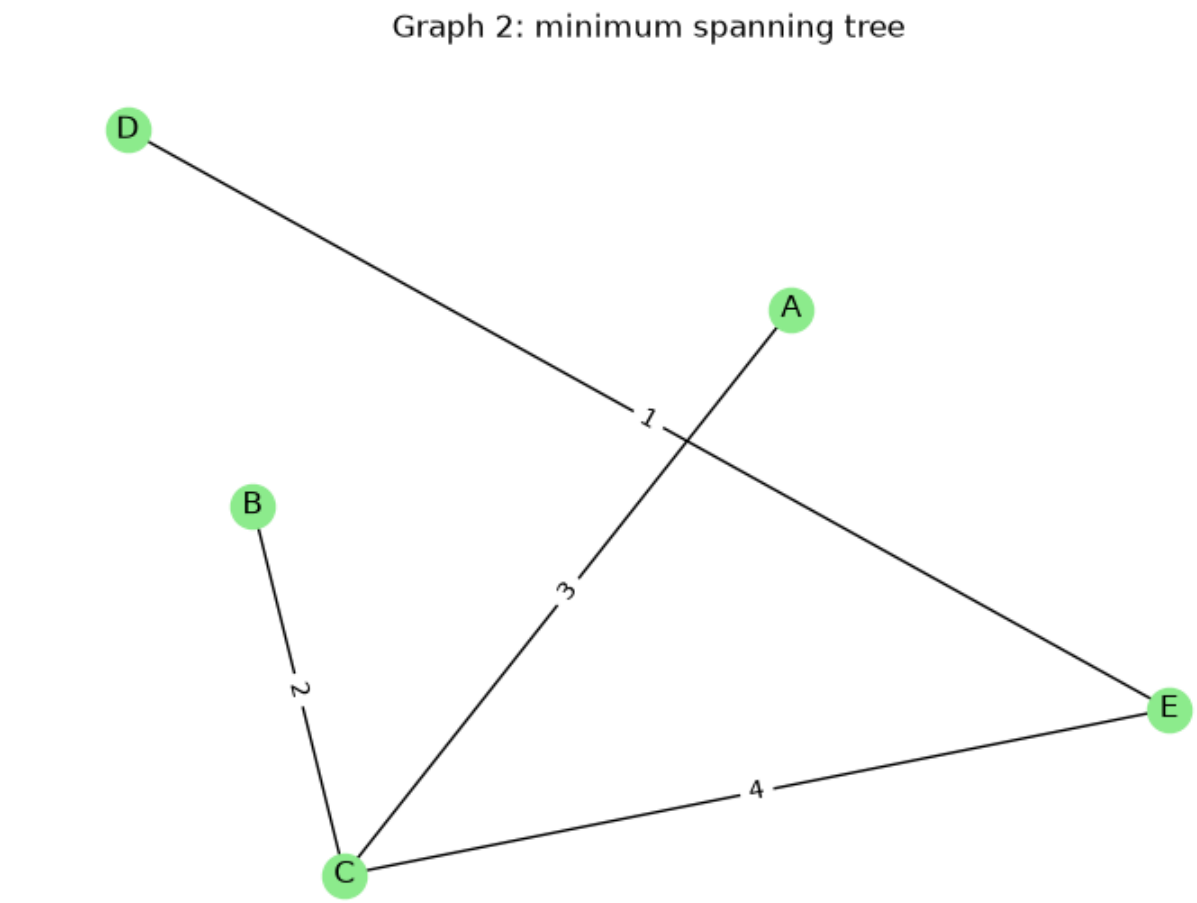
```
mst2 = MyMinimumSpanningTree(graph2)
```

```
Starting node: A
Added edge: A - C Weight: 3
Added edge: C - B Weight: 2
Added edge: C - E Weight: 4
Added edge: E - D Weight: 1
```

Display the minimum spanning tree for Graph 2

I draw "mst2" with its edge weights where the nodes use the same positions as Graph 2 so the diagrams are easier to compare.

```
plt.figure(figsize=(7, 5))
nx.draw(mst2, positions2, with_labels=True, node_color="lightgreen")
weights2 = nx.get_edge_attributes(mst2, "weight")
nx.draw_networkx_edge_labels(mst2, positions2, edge_labels=weights2)
plt.title("Graph 2: minimum spanning tree")
plt.axis("off")
plt.show()
```



I check that the tree has 5 nodes and 4 edges, is connected and has no cycles.

A loop adds the selected edge weights and the expected total is 10.

```
print("Number of nodes:", mst2.number_of_nodes())
print("Number of edges:", mst2.number_of_edges())
print("Is a tree:", nx.is_tree(mst2))
total_weight2 = 0

for node1, node2 in mst2.edges():
    weight = mst2[node1][node2]["weight"]
    total_weight2 = total_weight2 + weight
print("Total weight:", total_weight2)
```

```
Number of nodes: 5
Number of edges: 4
Is a tree: True
Total weight: 10
```

Testing: Graph 3

I test the same function on a third graph with 6 nodes and 9 weighted edges where you can see that its connections and weights differ from the first two graphs.

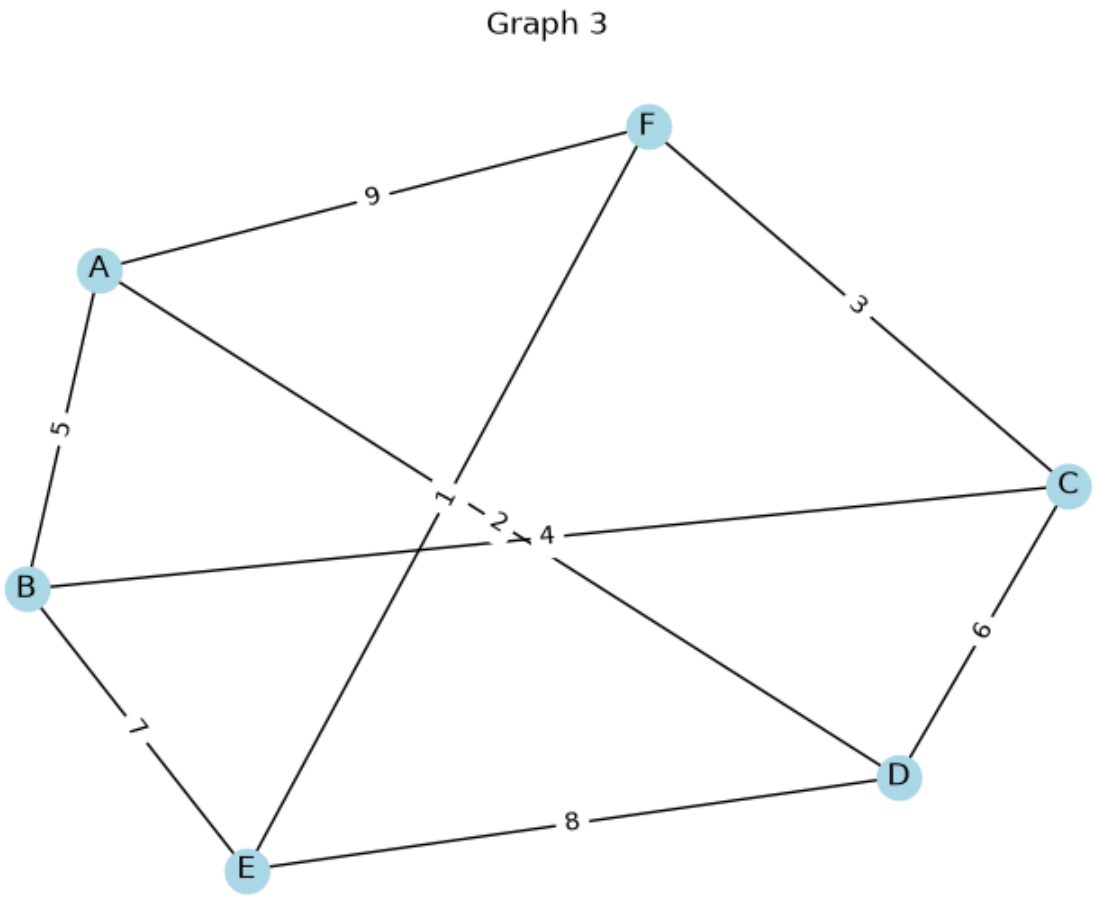
```
graph3 = nx.Graph()
edges3 = [
    ("A", "B", 5),
    ("A", "D", 2),
    ("A", "F", 9),
    ("B", "C", 4),
    ("B", "E", 7),
    ("C", "D", 6),
    ("C", "F", 3),
    ("D", "E", 8),
    ("E", "F", 1)
]
for node1, node2, weight in edges3:
    graph3.add_edge(node1, node2, weight=weight)
print("Number of nodes:", graph3.number_of_nodes())
print("Number of edges:", graph3.number_of_edges())
print("Graph is connected:", nx.is_connected(graph3))
```

```
Number of nodes: 6
Number of edges: 9
Graph is connected: True
```

Display Graph 3

I display the graph with its node labels and edge weights and save the node positions in "positions3" to use again when drawing its minimum spanning tree.

```
positions3 = nx.spring_layout(graph3, seed=1)
plt.figure(figsize=(7, 5))
nx.draw(graph3, positions3, with_labels=True, node_color="lightblue")
weights3 = nx.get_edge_attributes(graph3, "weight")
nx.draw_networkx_edge_labels(graph3, positions3, edge_labels=weights3)
plt.title("Graph 3")
plt.axis("off")
plt.show()
```



Run Prim's algorithm on Graph 3

I call the same function using "graph3" and the result is stored in "mst3" where the function prints each edge it adds and its weight.

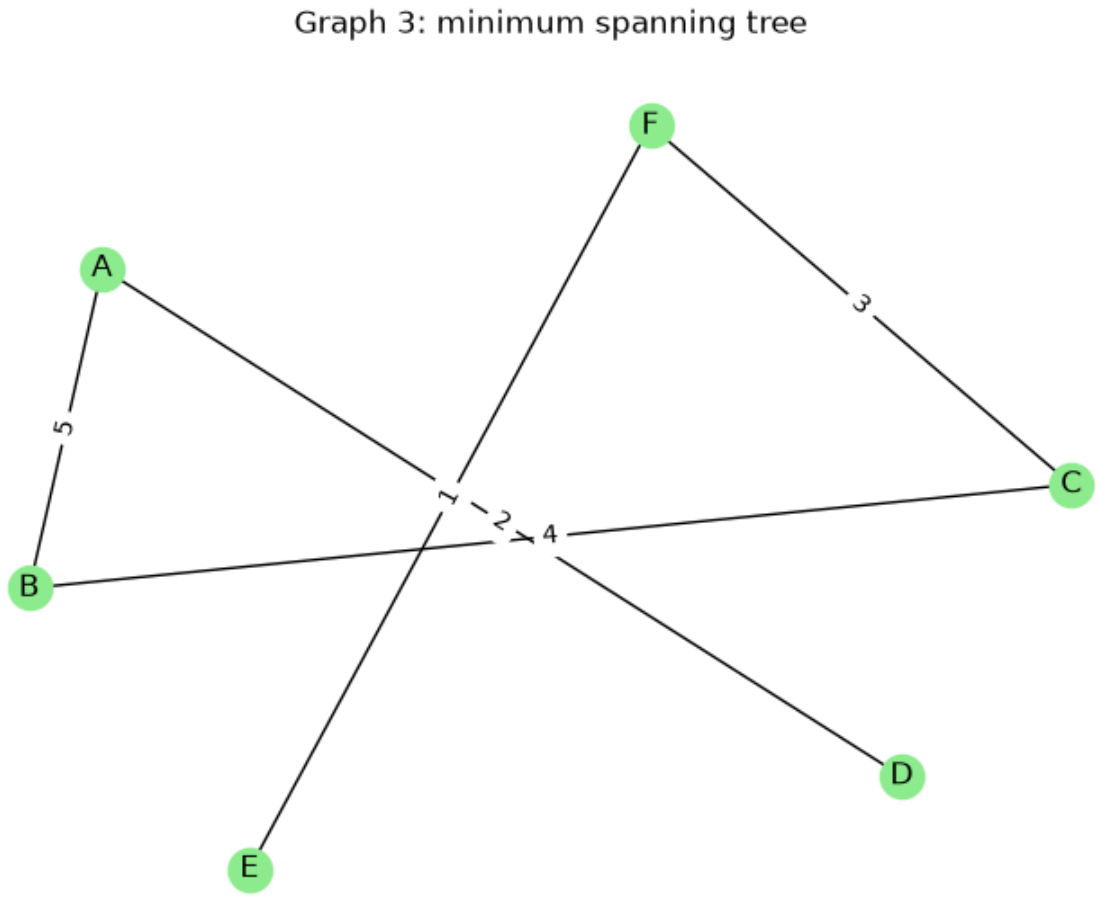
```
mst3 = MyMinimumSpanningTree(graph3)
```

```
Starting node: A
Added edge: A - D Weight: 2
Added edge: A - B Weight: 5
Added edge: B - C Weight: 4
Added edge: C - F Weight: 3
Added edge: F - E Weight: 1
```

Display the minimum spanning tree for Graph 3

I draw "mst3" with its edge weights where nodes use the same positions as Graph 3 so the diagrams are easier to compare.

```
plt.figure(figsize=(7, 5))
nx.draw(mst3, positions3, with_labels=True, node_color="lightgreen")
weights3 = nx.get_edge_attributes(mst3, "weight")
nx.draw_networkx_edge_labels(mst3, positions3, edge_labels=weights3)
plt.title("Graph 3: minimum spanning tree")
plt.axis("off")
plt.show()
```



I check that the tree has 6 nodes and 5 edges, is connected and has no cycles

A for loop adds the selected edge weights which gives the expected total of 15.

```
print("Number of nodes:", mst3.number_of_nodes())
print("Number of edges:", mst3.number_of_edges())
print("Is a tree:", nx.is_tree(mst3))

total_weight3 = 0
for node1, node2 in mst3.edges():
    weight = mst3[node1][node2]["weight"]
    total_weight3 = total_weight3 + weight
print("Total weight:", total_weight3)
```

```
Number of nodes: 6
Number of edges: 5
Is a tree: True
Total weight: 15
```

Results

Graph	Nodes	Original edges	Tree edges	Total weight
Graph 1	6	11	5	21
Graph 2	5	8	4	10
Graph 3	6	9	5	15

The same function produced a tree for all three graphs where each tree connects all the original nodes without a cycle.

Prim's algorithm uses a greedy approach because at each step, it selects the cheapest edge connecting the current tree to a new node and this means the choice is safe because a cheapest edge between the included nodes and the remaining nodes can be part of a minimum spanning tree (Sedgewick and Wayne, no date).

Repeating this step gives a tree with the lowest total weight and the tests confirm the tree structure and calculate its weight. However these checks alone do not prove that the weight is minimal.

Data structures used

The NetworkX graph stores the nodes, edges and weights where the edges are stored in a list of tuples and each tuple contains two nodes and their edge weight. A for loop adds them to the NetworkX graph, which stores the nodes, edges and weights.

The "visited" list tracks the nodes already included and an if statement checks whether a neighbouring node has not yet been added.

The variable "tree" stores the minimum spanning tree being built and then the function returns it when all nodes have been included.

References

Moros, S. (no date) 'Unit 4: Algorithm programming: graph algorithms'. *7WCM2026: Data Structures and Algorithms 2*. University of Hertfordshire. Available at: https://herts.instructure.com/courses/130423/pages/unit-4-algorithm-programming-graph-algorithms (Accessed: 19 October 2026).

Sedgewick, R. and Wayne, K. (no date) *Minimum spanning trees*. Available at: https://algs4.cs.princeton.edu/43mst/ (Accessed: 19 October 2026).